Arquitectura de Codigo

Simulador de Pensiones IMSS + Fondo Bienestar

Documento 3 de 4 — Serie de Referencia Tecnica

Para uso personal de referencia

Andres Tejada

Febrero 2026

Este documento mapea la arquitectura completa del simulador de pensiones: jerarquia de dependencias, motor de calculos actuariales, grafo reactivo de Shiny, sistema de componentes UI, pipeline de generacion de reportes y arquitectura de pruebas. Cada seccion utiliza el sistema de tres cajas (patron formal, intuicion de flujo y referencia de codigo) para maxima profundidad.

Contenido

1 Vision General del Sistema **4**
1.1 Inventario de Archivos . . . 4
1.2 Jerarquia de Dependencias y Orden de Carga . . . 5
1.2.1 Diagrama de Dependencias . . . 6
1.2.2 Patron de Doble Carga de Librerias . . . 6
1.3 Constantes y Valores Regulatorios . . . 7

2 Capa de Datos **8**
2.1 Archivos CSV . . . 8
2.2 Funciones de Acceso a Datos . . . 8
2.2.1 lookup_articulo_167 . . . 9
2.2.2 get_esperanza_vida . . . 9
2.2.3 Funciones de Validacion . . . 10
2.2.4 Tabla Completa de Funciones de Datos . . . 11

3 Motor de Calculos Actuariales **12**
3.1 Ley 73: Pension Definida . . . 12
3.1.1 calculate_ley73_pension . . . 12
3.1.2 Diagrama de la Cadena Ley 73 . . . 14
3.2 Ley 97: Cuenta Individual AFORE . . . 14
3.2.1 project_afore_balance . . . 14
3.2.2 calculate_aportacion_obligatoria . . . 15
3.2.3 calculate_retiro_programado . . . 16
3.2.4 calculate_ley97_pension (Pipeline Completo) . . . 17
3.2.5 Diagrama del Pipeline Ley 97 . . . 17
3.3 Modalidad 40 . . . 17
3.4 Comparacion de AFOREs . . . 18
3.5 Tabla Completa de Funciones del Motor . . . 18

4 Modulo Fondo Bienestar **20**
4.1 Elegibilidad . . . 20
4.2 Calculo del Complemento . . . 21
4.3 Orquestador de 3 Escenarios . . . 21
4.3.1 Diagrama de los 3 Escenarios . . . 23
4.4 Funciones de Sensibilidad y Mensajes . . . 23
4.5 Tabla de Funciones del Modulo Fondo . . . 23

5 Arquitectura UI: Componentes y Diseno **24**
5.1 Sistema de Diseno CSS . . . 24
5.1.1 Breakpoints Responsivos . . . 25
5.2 Componentes R de UI . . . 25
5.2.1 pension_theme . . . 25
5.2.2 Landing Page Components . . . 26
5.2.3 Wizard Components . . . 26
5.2.4 Result Rendering . . . 26
5.2.5 Componentes Auxiliares de UI . . . 27
5.3 Tabla de Componentes UI . . . 27

6 Flujo Reactivo del Servidor **29**
6.1 Estado Reactivo . . . 29
6.2 Transiciones de Estado del Wizard . . . 30
6.3 Auto-fill y Deteccion de Regimen . . . 30
6.4 Calculo Principal . . . 31
6.5 Sensibilidad con Debounce . . . 32
6.6 Impact Labels (Etiquetas de Impacto) . . . 33
6.7 Cliff Notifications . . . 34
6.8 Grafica Plotly . . . 34
6.9 Diagrama del Grafo Reactivo Completo . . . 36

7 Pipeline de Generacion de Documentos **37**
7.1 Arquitectura de Reportes . . . 37
7.2 Ciclo de Vida de Archivos Temporales . . . 38
7.2.1 Diagrama del Pipeline de Reportes . . . 38
7.3 Tabla de Funciones de Documentos . . . 38

8 Arquitectura de Pruebas **40**
8.1 Estructura del Test Suite . . . 40
8.2 Secciones de Prueba . . . 41
8.3 Patron expect_num . . . 41
8.4 Otros Archivos de Prueba . . . 42

A Indice Completo de Funciones **43**
A.1 global.R (153 lineas) . . . 43
A.2 R/data_tables.R (244 lineas) . . . 43
A.3 R/calculations.R (671 lineas) . . . 43
A.4 R/fondo_bienestar.R (497 lineas) . . . 44

Capítulo 1

Vision General del Sistema

1.1 Inventario de Archivos

El sistema consiste en 9 archivos fuente principales, 4 archivos de datos CSV, 2 documentos de referencia y 6 archivos de pruebas.

Patron de Software

Patron: Modular Monolith. El simulador sigue una arquitectura de monolito modular: un solo proceso R sirve toda la aplicacion, pero la logica esta separada en archivos con responsabilidades claras. No hay framework MVC formal, pero la separacion implicita es: `R/data_tables.R` (Modelo/Datos), `R/calculations.R` + `R/fondo_bienestar.R` (Logica de Negocio), `R/ui_helpers.R` (Vista), y `app.R` (Controlador/Server).

Intuicion de Flujo

Piensa en el sistema como un pipeline lineal: los datos entran por la izquierda (CSVs), se transforman en el centro (calculos actuariales), y salen por la derecha (HTML renderizado). Los datos fluyen hacia adelante; el estado reactivo de Shiny es lo que permite que el pipeline se re-ejecute cuando el usuario mueve un slider.

Referencia de Codigo

Archivos principales y lineas de codigo:

- `app.R` — 1,951 lineas (UI: 1–737, Server: 743–1,951)
- `global.R` — 153 lineas (carga de paquetes, datos, constantes)
- `R/calculations.R` — 671 lineas (motor actuarial)
- `R/fondo_bienestar.R` — 497 lineas (complemento Fondo)
- `R/data_tables.R` — 244 lineas (consulta de datos)
- `R/ui_helpers.R` — 1,330 lineas (componentes UI)
- `R/document_generators.R` — 1,667 lineas (reportes HTML/PDF)
- `www/styles.css` — 2,156 lineas (sistema de diseno)
- `tests/testthat/test_calculations.R` — 1,315 lineas (95 pruebas)

Total: ∼10,000 lineas de codigo (sin contar CSV ni docs).

1.2 Jerarquia de Dependencias y Orden de Carga

Patron de Software

Patron: Eager Loading with Global Scope. El sistema carga todos los datos y funciones en el arranque, usando superasignacion (`«-`) para inyectar valores en el ambiente global de R. Esto es un patron deliberado de Shiny: las funciones definidas en archivos `source()`-eados dentro de `global.R` necesitan ser visibles para la funcion `server()` en `app.R`.

Intuicion de Flujo

El flujo de carga es como una cascada: `app.R` dispara `global.R`, que primero lee los archivos CSV (dato crudo), luego define constantes (datos procesados), y finalmente carga las funciones R en orden de dependencia. **Orden importa**: `data_tables.R` debe cargarse antes que `calculations.R` porque las funciones de calculo llaman a `lookup_articulo_167()`.

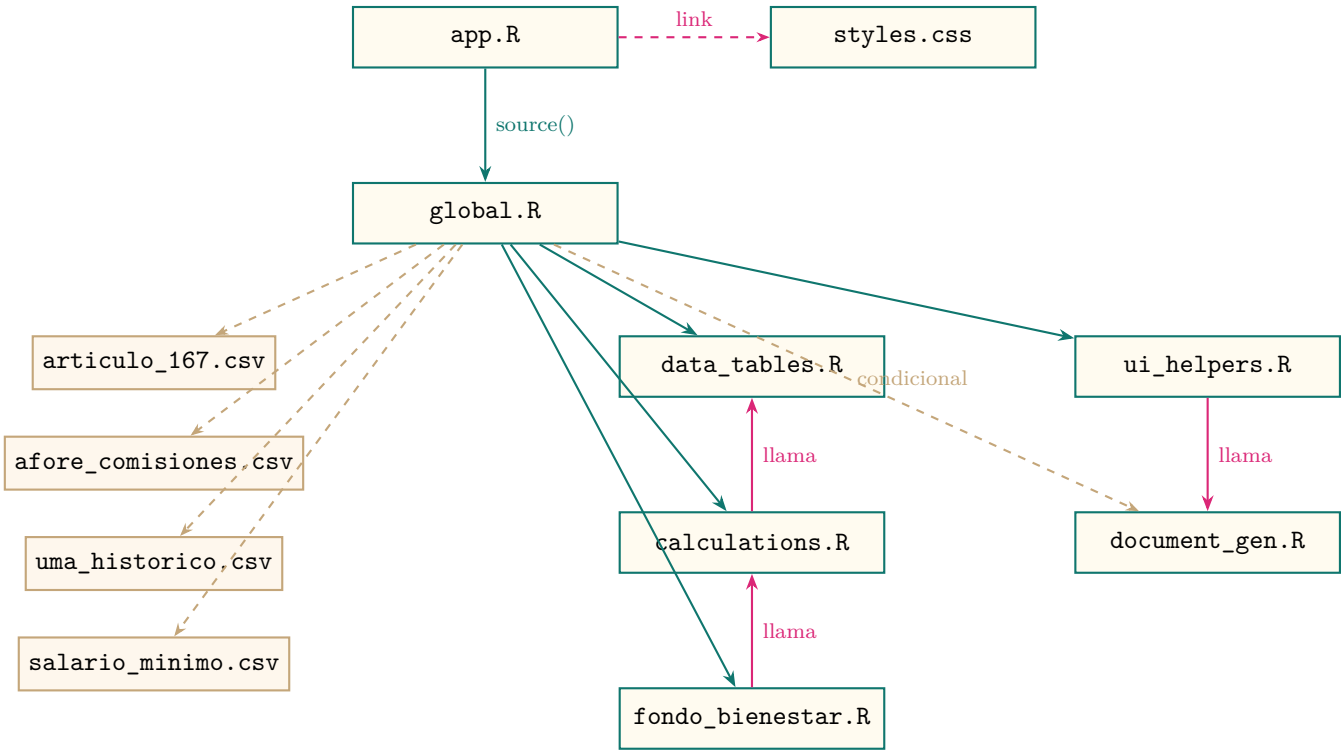
Referencia de Codigo

Cadena de carga en `global.R` lineas 29–152:

```r
# Datos (lineas 29-38)
articulo_167_tabla <<- read.csv("data/articulo_167_tabla.csv")
uma_data <<- read.csv("data/uma_historico.csv")
salario_minimo_data <<- read.csv("data/salario_minimo.csv")
afore_data <<- read.csv("data/afore_comisiones.csv")

# Constantes (lineas 44-74)
UMA_DIARIA_2025 <<- 113.14
SM_DIARIO_2025 <<- 278.80
UMBRAL_FONDO_BIENESTAR_2025 <<- 17364
FACTORES_CESANTIA <<- c("60"=0.75, "61"=0.80, ..., "65"=1.00)

# Funciones utilitarias (lineas 84-138)
`%||%` <<- function(x, y) if (is.null(x) || length(x)==0) y
    else x
format_currency <<- function(x) paste0("$", format(round(x,2),
    big.mark=",", nsmall=2))

# Source files (lineas 144-152)
source("R/data_tables.R")     # Primero: datos
source("R/calculations.R")    # Segundo: motor actuarial
source("R/fondo_bienestar.R")# Tercero: fondo (depende de
    calculations)
source("R/ui_helpers.R")      # Cuarto: UI
if (file.exists("R/document_generators.R")) source("R/
    document_generators.R") # Condicional
```

Listing 1.1: Orden de carga en global.R

1.2.1 Diagrama de Dependencias



1.2.2 Patron de Doble Carga de Librerias

Patron de Software

Patron: Dual Library Loading. Las librerias se cargan en `global.R` (lineas 17–22) Y tambien se referencian con namespace en `app.R` (por ejemplo `plotly::plotlyOutput`). Esto es necesario porque la definicion de UI en Shiny se evalua antes de que `server()` se ejecute, y expresiones como `plotlyOutput()` necesitan que el paquete `plotly` este disponible en tiempo de evaluacion de UI.

Referencia de Codigo

`global.R` linea 20: `library(plotly)` — carga el paquete.
`app.R` dentro de UI: `plotly::plotlyOutput("pension_chart")` — referencia explicita con namespace para evitar ambiguedad.
Si omites el `library()` en `global.R`, la UI falla al renderizar porque `plotlyOutput` no existe aun cuando Shiny evalua la definicion de UI.

1.3 Constantes y Valores Regulatorios

Intuicion de Flujo

Todas las constantes regulatorias viven en un solo lugar: `global.R` lineas 44–74. Para actualizar el simulador al ano 2026, solo necesitas cambiar ∼5 valores en este bloque. Los umbrales del Fondo Bienestar tienen su propia funcion `get_umbral_fondo_bienestar()` con valores historicos hardcodeados.

Tabla 1.1: Constantes regulatorias definidas en `global.R`

Variable	Valor	Linea	Fuente
`ANIO_ACTUAL`	2025	45	—
`UMA_DIARIA_2025`	113.14	48	INEGI/DOF
`UMA_MENSUAL_2025`	3,439.46	49	Calculado
`SM_DIARIO_2025`	278.80	52	CONASAMI
`SM_MENSUAL_2025`	8,474.52	53	Calculado
`UMBRAL_FONDO_2025`	17,364	56	DOF/IMSS
`TOPE_SBC_DIARIO`	2,828.50	59	25 × UMA
`RENDIMIENTO_BASE`	0.04 (4%)	63	—
`FACTORES_CESANTIA`	0.75–1.00	67–74	LSS Art. 167

Capítulo 2

Capa de Datos

2.1 Archivos CSV

Patron de Software

Patron: Static Lookup Tables. Los datos regulatorios se almacenan como archivos CSV planos, cargados una sola vez al arranque via `read.csv()` con superasignacion `«-`. No hay base de datos, no hay API — todo es lectura en memoria. Esto es apropiado porque los datos regulatorios cambian como maximo una vez al ano.

Intuicion de Flujo

Los 4 CSV son "tablas de consulta" (lookup tables). El flujo es: **usuario ingresa salario** → se divide entre salario minimo → se busca en `articulo_167_tabla.csv` → se obtiene cuantia basica. Es un patron tipico de datos actuariales donde las formulas dependen de rangos discretos definidos por ley.

Referencia de Codigo

Archivos de datos y sus estructuras:

- `data/articulo_167_tabla.csv`: 24 filas, columnas `grupo_min`, `grupo_max`, `cuantia_basica`, `incremento_anual`. Leido en `global.R:29`.
- `data/afore_comisiones.csv`: 11 filas (una por AFORE), columnas `afore`, `comision_2024`, `comision_2025`, `irn_2024`. Leido en `global.R:38`.
- `data/uma_historico.csv`: 10 filas (2016–2025), columnas `anio`, `uma_diaria`. Leido en `global.R:32`.
- `data/salario_minimo.csv`: 10 filas (2016–2025), columnas `anio`, `sm_diario`. Leido en `global.R:35`.

2.2 Funciones de Acceso a Datos

El archivo `R/data_tables.R` (244 lineas) provee la capa de abstraccion sobre los datos crudos. Contiene 8 funciones publicas.

2.2.1 lookup_articulo_167

Patron de Software

Patron: Range Lookup. A diferencia de un lookup por clave exacta, esta funcion busca en *rangos*: el grupo salarial debe caer entre `grupo_min` y `grupo_max` de alguna fila. Si excede el maximo, usa la ultima fila; si es menor al minimo, usa la primera. Es un patron defensivo que garantiza que siempre se retorna un valor.

Intuicion de Flujo

El articulo 167 de la LSS define 24 grupos salariales. Cada grupo tiene un "porcentaje base" (cuantia basica) y un "incremento por ano extra" (incremento anual). Un trabajador que gana 3 veces el salario minimo cae en un grupo diferente que uno que gana 6 veces. La tabla es fija por ley — no cambia con inflacion.

Referencia de Codigo

`R/data_tables.R` lineas 12–30:

```r
lookup_articulo_167 <- function(grupo_salarial, tabla =
    articulo_167_tabla) {
  idx <- which(grupo_salarial >= tabla$grupo_min &
      grupo_salarial <= tabla$grupo_max)
  if (length(idx) == 0) {
    if (grupo_salarial > max(tabla$grupo_max)) {
      idx <- nrow(tabla)        # Tope superior
    } else {
      idx <- 1                  # Piso inferior
    }
  }
  return(list(
    cuantia_basica = tabla$cuantia_basica[idx],
    incremento_anual = tabla$incremento_anual[idx]
  ))
}
```

Listing 2.1: lookup_articulo_167

2.2.2 get_esperanza_vida

Patron de Software

Patron: Interpolation with Boundary Clamping. Para edades que no aparecen exactamente en la tabla (como 72 o 77 anos), la funcion interpola linealmente entre los puntos conocidos. Para edades fuera de rango, aplica extrapolacion con limites: por debajo del minimo, suma anos directamente; por encima del maximo, decrementa con un piso de 2 anos.

Intuicion de Flujo

Las tablas de mortalidad CONAPO definen puntos a edades clave (60, 65, 70, 75, 80, 85, 90). Si el usuario tiene 73 anos, necesitamos un valor entre los puntos 70 y 75. La interpolacion lineal es una simplificacion educativa — las tablas actuariales reales usan funciones de Gompertz-Makeham. Mujeres viven ∼5 anos mas que hombres en la tabla, lo que significa menores pensiones mensuales bajo retiro programado (mismo saldo, mas meses de vida).

Referencia de Codigo

`R/data_tables.R` lineas 89–143. Estructura interna:

```r
get_esperanza_vida <- function(edad, genero = "M") {
  # Validacion de NULLs (lineas 94-99)
  if (genero == "F") {
    base <- c("60"=24.5, "65"=20.0, "70"=15.9, "75"=12.5, "80"=9.5, ...)
  } else {
    base <- c("60"=21.0, "65"=17.0, "70"=13.4, "75"=10.5, "80"=8.0, ...)
  }
  # Lookup exacto (lineas 117-120)
  if (edad_str %in% names(base)) return(base[edad_str])
  # Interpolacion lineal (lineas 133-142)
  prop <- (edad - edades[idx_inf]) / (edades[idx_sup] - edades[idx_inf])
  return(valores[idx_inf] + prop * (valores[idx_sup] - valores[idx_inf]))
}
```

Listing 2.2: Tablas de mortalidad con interpolacion

2.2.3 Funciones de Validacion

Patron de Software

Patron: Structured Validation Response. Las funciones de validacion retornan listas con estructura fija: `{valido, errores, advertencias}` para `validar_entrada()`, y `{is_consistent, message}` para `validar_consistencia_fechas()`. Esto permite al server de Shiny procesar errores y warnings por separado.

Referencia de Codigo

Funciones de validacion en `R/data_tables.R`:

- `validar_entrada(edad, semanas, sbc, fecha)` — lineas 155–203. Valida: edad 18–90, semanas ≥ 0 (warn > 3000), SBC ≥ 0, consistencia edad-semanas (semanas $\leq$ (edad $-$ 18) $\times$ 52), tope de cotizacion (25 UMA).
- `determinar_regimen(fecha)` — lineas 208–222. Retorna `"ley73"` si fecha $<$ 1997-07-01, `"ley97"` si no. Convierte string a Date si necesario.

- `validar_consistencia_fechas(nac, inicio)` — lineas 228–244. Calcula edad de inicio de cotizacion; warn si < 15 o > 35.

2.2.4 Tabla Completa de Funciones de Datos

Tabla 2.1: Inventario de funciones en `R/data_tables.R`

Funcion	**Linea**	**Retorna**	**Proposito**
`lookup_articulo_167`	12	list	Busqueda por rango en tabla Art. 167
`get_afore_names`	38	character	Nombres de 11 AFOREs
`get_afore_comision`	46	numeric	Comision con fallback a promedio
`get_afore_irn`	63	numeric	IRN con fallback a promedio
`get_all_afores`	73	data.frame	Datos completos de AFOREs
`get_esperanza_vida`	89	numeric	Esperanza de vida interpolada
`validar_entrada`	155	list	Validacion de inputs del usuario
`determinar_regimen`	208	character	Deteccion de regimen por fecha
`validar_consistencia_fechas`	228	list	Validacion cruzada de fechas

Capítulo 3

Motor de Calculos Actuariales

El archivo `R/calculations.R` (671 lineas) contiene las 8 funciones que implementan las formulas actuariales del IMSS. Este es el corazon matematico del simulador.

3.1 Ley 73: Pension Definida

3.1.1 calculate_ley73_pension

Patron de Software

Patron: Sequential Pipeline with Early Returns. La funcion implementa una cadena de 8 pasos secuenciales, con dos salidas tempranas (early returns) para elegibilidad: semanas < 500 y edad < 60. Cada paso depende del anterior, y el resultado final es una lista comprehensiva que incluye tanto el valor final como todos los valores intermedios (para transparencia en la UI).

Intuicion de Flujo

La formula de Ley 73 es una cadena multiplicativa:

$$\text{pension} = \text{SBC} \times \underbrace{(\text{cuantia} + n \times \text{incremento})}_{\text{de Art. 167}} \times \underbrace{\text{factor_edad}}_{0.75\text{–}1.00}$$

Luego se aplica un piso: la pension nunca es menor a 1 salario minimo mensual. El factor de edad es lo que penaliza retirarse antes de los 65: a los 60 pierdes 25% de tu pension. Por eso el slider de edad de retiro tiene tanto impacto.

Referencia de Codigo

`R/calculations.R` lineas 21–103. Los 8 pasos:

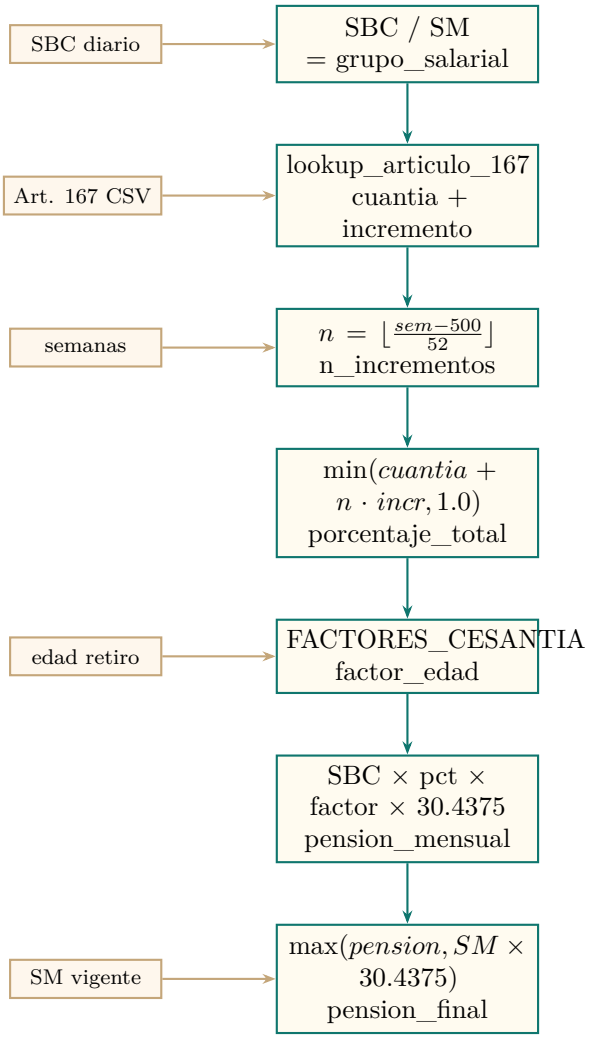
```
calculate_ley73_pension <- function(sbc_promedio_diario,
    semanas, edad,
                                    sm_vigente =
                                        SM_DIARIO_2025,
```

```r
                                                tipo_pension = "vejez")
                                                 {
  # Early returns: semanas < 500 (linea 28), edad < 60 (linea 36)
  if (semanas < 500) return(list(pension_mensual=0, elegible=FALSE, ...))
  if (edad < 60) return(list(pension_mensual=0, elegible=FALSE, ...))

  # Paso 1: Grupo salarial (linea 45)
  grupo_salarial <- sbc_promedio_diario / sm_vigente

  # Paso 2: Busqueda Art. 167 (linea 48)
  tabla_lookup <- lookup_articulo_167(grupo_salarial)

  # Paso 3: Incrementos (linea 53)
  n_incrementos <- max(0, floor((semanas - 500) / 52))

  # Paso 4: Porcentaje total con tope 100% (linea 57)
  porcentaje_total <- min(cuantia_basica + total_incrementos, 1.0)

  # Paso 5: Factor de edad/cesantia (lineas 60-67)
  factor_edad <- if (edad >= 65) 1.0 else FACTORES_CESANTIA[as.character(edad)]

  # Paso 6: Pension diaria -> mensual (lineas 72-75)
  DIAS_POR_MES <- 30.4375  # 365.25/12, estandar actuarial
  pension_mensual <- sbc_promedio_diario * porcentaje_total * factor_edad * DIAS_POR_MES

  # Paso 7: Pension minima (lineas 78-79)
  pension_final <- max(pension_mensual, sm_vigente * DIAS_POR_MES)

  # Paso 8: Return comprehensivo (lineas 85-102)
}
```

Listing 3.1: Cadena de Ley 73

Nota critica: Se usa 30.4375 dias/mes (no 30). La diferencia de 1.44% se acumula a lo largo de decadas y afecta la precision del calculo.

3.1.2 Diagrama de la Cadena Ley 73



3.2 Ley 97: Cuenta Individual AFORE

3.2.1 project_afore_balance

Patron de Software

Patron: Closed-Form with Optional Simulation. La funcion calcula el saldo futuro usando formulas cerradas (valor futuro de lump sum + anualidad), pero opcionalmente puede generar una trayectoria ano-por-ano usando un loop. La trayectoria se usa para la grafica de Plotly en Step 4; el valor cerrado es mas rapido para sensibilidad.

Intuicion de Flujo

El saldo futuro tiene dos componentes: (1) lo que ya tienes crece con interes compuesto: $FV_{saldo} = S_0 \times (1+r)^n$, y (2) lo que aportas cada mes crece como anualidad: $FV_{aport} = A \times \frac{(1+r_m)^{12n}-1}{r_m}$. La comision se resta del rendimiento: $r_{neto} = r_{real} - comision$. Es una

simplificacion; en realidad la comision se cobra sobre el saldo, no se resta del rendimiento. Pero para fines educativos la diferencia es menor.

Referencia de Codigo

`R/calculations.R` lineas 118–181:

```
1  project_afore_balance <- function(saldo_actual, aportacion_mensual,
2                                    anios_al_retiro, rendimiento_real_anual = 0.04,
3                                    comision_anual = 0.0053, incluir_trayectoria = FALSE) {
4    r_neto <- rendimiento_real_anual - comision_anual    # linea 126
5    r_mensual <- (1 + r_neto)^(1/12) - 1                 # linea 129
6    fv_actual <- saldo_actual * (1 + r_neto)^anios_al_retiro  # linea 133
7    # Caso especial: rendimiento cero (linea 136)
8    if (abs(r_mensual) < 1e-10) {
9      fv_aportaciones <- aportacion_mensual * meses
10   } else {
11     fv_aportaciones <- aportacion_mensual * ((1+r_mensual)^meses - 1) / r_mensual
12   }
13   saldo_final <- fv_actual + fv_aportaciones           # linea 143
14   # Trayectoria opcional (lineas 147-165): loop anual con compounding mensual
15 }
```

Listing 3.2: Proyeccion AFORE con formula cerrada

3.2.2 calculate_aportacion_obligatoria

Patron de Software

Patron: Year-Indexed Rate Table. La reforma 2020 establece un incremento gradual de la tasa patronal de 6.20% (2023) a 12.00% (2030). La funcion usa un vector nombrado para hacer lookup por ano, con fallback a 2025 si el ano no existe.

Referencia de Codigo

`R/calculations.R` lineas 188–235:

```
1  calculate_aportacion_obligatoria <- function(salario_mensual, anio = 2025) {
2    tasas_patron <- c(
```

```
    "2023"=0.0620, "2024"=0.0690, "2025"=0.0775, "2026"=0.0860,
    "2027"=0.0945, "2028"=0.1030, "2029"=0.1115, "2030"=0.1200)
  tasa_trabajador <- 0.01125      # Fijo
  tasa_gobierno <- 0.00225        # Aproximado
  tasa_total <- tasa_patron + tasa_trabajador + tasa_gobierno
  salario_cotizable <- min(salario_mensual, TOPE_SBC_DIARIO *
     30)
  aportacion <- salario_cotizable * tasa_total
}
```

Listing 3.3: Tasas de aportacion con reforma 2020

Simplificacion conocida: El codigo usa la tasa 2025 para todos los anos de proyeccion. La reforma per-year no se aplica aun.

3.2.3 calculate_retiro_programado

Patron de Software

Patron: Floor Guarantee. La pension por retiro programado es simplemente saldo/(esperanza_vida $\times$ 12), pero con un piso (floor): la pension minima garantizada de 2.5 UMA mensuales ($\sim$\$8,599 en 2025). Cuando el piso se activa, enmascara diferencias por genero y edad, ya que todos reciben lo mismo.

Referencia de Codigo

`R/calculations.R` lineas 259–298:

```
calculate_retiro_programado <- function(saldo, edad, genero = "
   M") {
  esperanza_vida <- get_esperanza_vida(edad, genero)  # linea
     262
  pension_calculada <- saldo / (esperanza_vida * 12)   # linea
     269
  pension_minima <- UMA_MENSUAL_2025 * 2.5             # linea
     277
  aplico_minimo <- FALSE
  if (pension_calculada < pension_minima) {
    pension_mensual <- pension_minima                   # linea
       283
    aplico_minimo <- TRUE
  }
}
```

Listing 3.4: Retiro programado con pension minima

Saldo necesario para superar minimo (hombre 65): $\$8,599 \times 204 = \$1,754,124$.

3.2.4 calculate_ley97_pension (Pipeline Completo)

Patron de Software

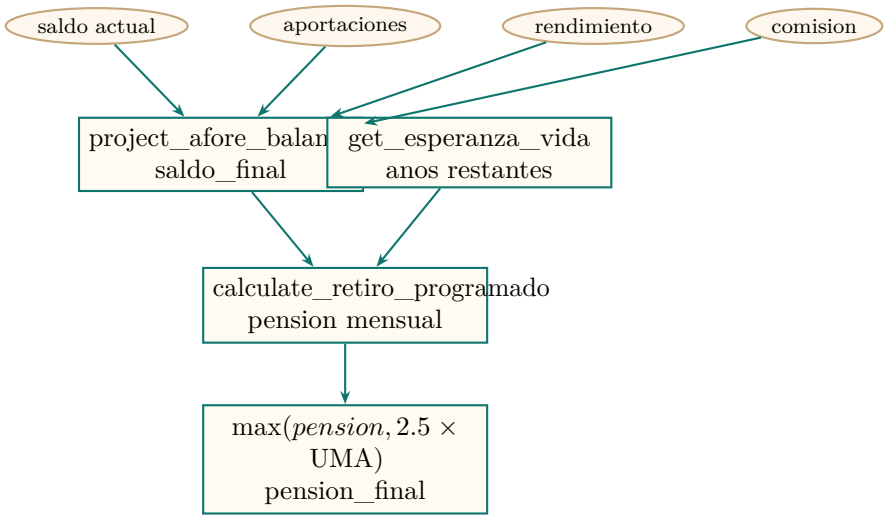
Patron: Orchestrator. Esta funcion es un orquestador que llama a `project_afore_balance()` y `calculate_retiro_programado()` en secuencia, agregando la logica de elegibilidad (1,000 semanas) y seleccion de escenario (conservador/base/optimista).

Referencia de Codigo

`R/calculations.R` lineas 313–391. Pipeline:

1. Validar semanas al retiro $\geq$ 1,000 (lineas 324–334)
2. Seleccionar rendimiento por escenario (lineas 337–342)
3. Obtener comision de AFORE (linea 345)
4. Calcular aportacion obligatoria (lineas 348–349)
5. Proyectar saldo: `project_afore_balance()` (lineas 352–359)
6. Calcular pension: `calculate_retiro_programado()` (lineas 362–366)
7. Calcular tasa de reemplazo (linea 369)
8. Return comprehensivo con 18 campos (lineas 371–391)

3.2.5 Diagrama del Pipeline Ley 97



3.3 Modalidad 40

Patron de Software

Patron: What-If Analysis. `calculate_modalidad_40()` recibe la pension actual como input y recalcula con parametros mejorados. Es un patron de analisis "que pasaria si": dado que ya calculaste tu pension base, que pasa si cotizas voluntariamente con un SBC mas alto?

Referencia de Codigo

`R/calculations.R` lineas 412–487. Logica clave:

```r
# Cuota mensual M40: 10.075% del SBC elegido (lineas 425-426)
cuota_mensual_m40 <- sbc_m40_real * 30 * 0.10075

# Nuevo SBC promedio ponderado (lineas 433-441)
if (semanas_m40 >= 250) {
  nuevo_sbc_promedio <- sbc_m40_real  # M40 cubre todo el promedio
} else {
  nuevo_sbc_promedio <- (sbc_m40 * semanas_nuevas + sbc_actual * semanas_viejas) / 250
}

# Recalcular pension Ley 73 con nuevos parametros (lineas 445-450)
nueva_pension <- calculate_ley73_pension(nuevo_sbc_promedio, semanas_totales, edad)

# ROI: meses para recuperar inversion (lineas 460-464)
meses_recuperacion <- ceiling(costo_total_m40 / incremento_pension)
```

Listing 3.5: Modalidad 40 con promedio ponderado

3.4 Comparacion de AFOREs

Referencia de Codigo

`R/calculations.R` lineas 624–671. `compare_afores()` itera sobre las 11 AFOREs, proyectando saldo con `project_afore_balance()` usando el IRN de cada una como proxy de rendimiento. Retorna un `data.frame` ordenado por saldo final, con la diferencia vs la AFORE actual del usuario.

3.5 Tabla Completa de Funciones del Motor

Tabla 3.1: Inventario de funciones en `R/calculations.R`

Funcion	**Linea**	**Params**	**Proposito**
`calculate_ley73_pension`	21	5	Cadena de 8 pasos Ley 73
`project_afore_balance`	118	6	Proyeccion FV con trayectoria
`calculate_aportacion_obligatoria`	188	2	Tasas de contribucion por ano
`calculate_retiro_programado`	259	3	Pension con piso 2.5 UMA
`calculate_ley97_pension`	313	9	Pipeline completo Ley 97
`calculate_modalidad_40`	412	7	Costo-beneficio M40
`calculate_all_scenarios`	498	9	Multi-escenario comparativo
`compare_afores`	624	4	Comparacion entre 11 AFOREs

Capítulo 4

Modulo Fondo Bienestar

El archivo `R/fondo_bienestar.R` (497 lineas) implementa la logica del Fondo de Pensiones para el Bienestar, decretado el 01/05/2024 (DOF) y en vigor desde el 01/07/2024.

4.1 Elegibilidad

Patron de Software

Patron: Checklist Validation. `check_fondo_eligibility()` evalua 4 condiciones independientes y retorna un reporte estructurado con cada condicion evaluada individualmente. Esto permite a la UI mostrar un checklist visual de requisitos cumplidos/no cumplidos, en lugar de un simple "si/no".

Intuicion de Flujo

El Fondo es exclusivo para Ley 97 porque los trabajadores de Ley 73 ya tienen pension definida (generalmente mejor). Los 4 filtros son: regimen correcto, edad $\geq$ 65, semanas $\geq$ 1000, y salario $\leq$ umbral ($17,364 en 2025). El umbral sube cada ano: $16,778 en 2024, $17,364 en 2025, ~$18,050 en 2026.

Referencia de Codigo

`R/fondo_bienestar.R` lineas 25–92:

```
check_fondo_eligibility <- function(regimen, edad, semanas,
                                    sbc_promedio_mensual, anio
                                        = ANIO_ACTUAL) {
  umbral <- get_umbral_fondo_bienestar(anio)
  checks <- list(
    regimen_valido    = list(cumple = (regimen == "ley97"),
        mensaje = ...),
    edad_minima       = list(cumple = (edad >= 65), mensaje =
        ...),
    semanas_minimas   = list(cumple = (semanas >= 1000),
        mensaje = ...),
```

```
    salario_bajo_umbral = list(cumple = (sbc_promedio_mensual <= umbral), mensaje = ...)
  )
  elegible <- all(sapply(checks, function(x) x$cumple))  # linea 72
  # Primera razon de no elegibilidad (lineas 76-83)
}
```

Listing 4.1: Checklist de elegibilidad del Fondo

4.2 Calculo del Complemento

Patron de Software

Patron: Gap-Fill Complement. El complemento no es un monto fijo — es la diferencia entre el "objetivo" (menor entre salario y umbral) y la pension AFORE. Matematicamente: complemento $= \max(0, \min(salario, umbral) - pension_{afore})$. Si la pension AFORE ya supera el objetivo, el complemento es cero.

Referencia de Codigo

`R/fondo_bienestar.R` lineas 107–145:

```
calculate_fondo_complement <- function(pension_afore, sbc_promedio_mensual, elegibilidad) {
  if (!elegibilidad$elegible) return(list(complemento = 0, ...))
  pension_objetivo <- min(sbc_promedio_mensual, elegibilidad$umbral_usado) # linea 128
  complemento <- max(0, pension_objetivo - pension_afore)  # linea 131
  pension_total <- pension_afore + complemento  # linea 133
}
```

Listing 4.2: Formula del complemento

4.3 Orquestador de 3 Escenarios

Patron de Software

Patron: Multi-Scenario Orchestrator. `calculate_pension_with_fondo()` es la funcion central del modulo. Calcula 3 escenarios en secuencia y retorna una lista anidada con toda la informacion para que la UI pueda renderizar el hero, el breakdown y las comparaciones.

Intuicion de Flujo

Los 3 escenarios responden a la pregunta clave del usuario:

1. **Solo sistema**: "Que pension me da el IMSS sin nada mas?"
2. **Con Fondo**: "Y si califico para el Fondo Bienestar?"
3. **Con acciones**: "Y si ademas ahorro voluntariamente?"

La narrativa del simulador empuja al usuario hacia el escenario 3: las aportaciones voluntarias son lo unico que el usuario controla directamente.

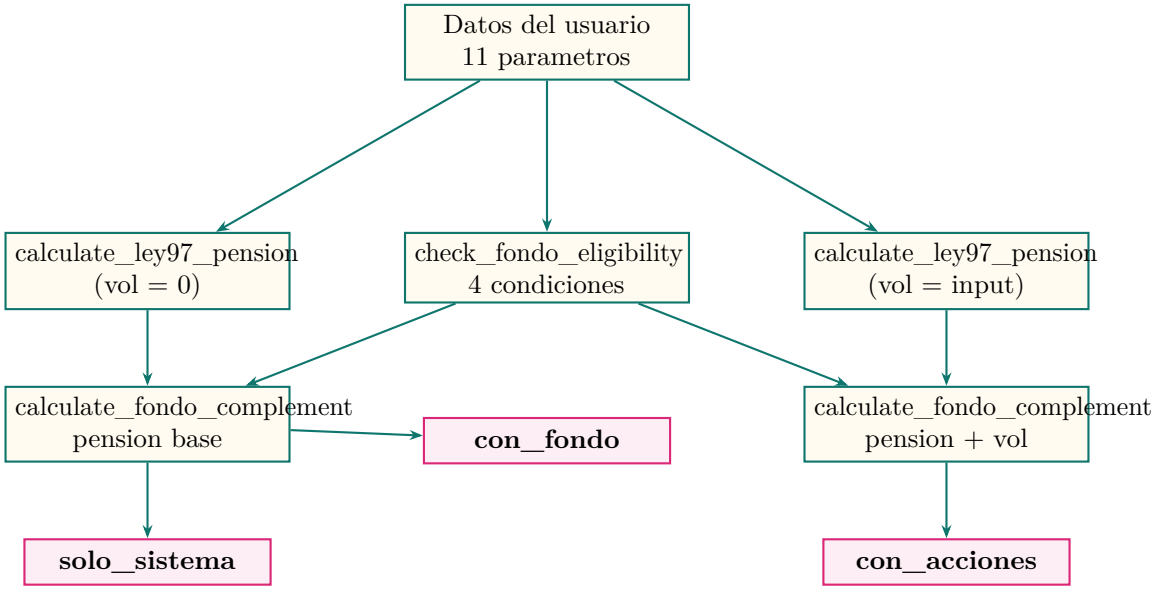
Referencia de Codigo

`R/fondo_bienestar.R` lineas 165–304. Estructura:

```r
calculate_pension_with_fondo <- function(saldo_actual, salario_mensual,
    edad_actual, edad_retiro=65, semanas_actuales, genero="M",
    aportacion_voluntaria=0, afore_nombre="XXI Banorte", escenario="base") {

  # Escenario 1: Solo sistema (lineas 179-189)
  pension_solo <- calculate_ley97_pension(..., aportacion_voluntaria = 0)

  # Elegibilidad del Fondo (lineas 196-201)
  elegibilidad <- check_fondo_eligibility("ley97", edad_retiro, semanas_al_retiro, ...)

  # Escenario 2: Con Fondo (lineas 204-208)
  complemento <- calculate_fondo_complement(pension_solo$pension_mensual, ...)

  # Escenario 3: Con acciones del usuario (lineas 211-228)
  pension_acciones <- calculate_ley97_pension(..., aportacion_voluntaria = aportacion_voluntaria)
  complemento_acciones <- calculate_fondo_complement(pension_acciones$pension_mensual, ...)

  # Return: lista anidada con solo_sistema, con_fondo, con_acciones, fondo_bienestar
  # (lineas 232-303)
}
```

Listing 4.3: Orquestador de 3 escenarios

4.3.1 Diagrama de los 3 Escenarios



4.4 Funciones de Sensibilidad y Mensajes

Referencia de Codigo

Funciones auxiliares en `R/fondo_bienestar.R`:

- `analyze_voluntary_contributions()` (lineas 315–356): Itera sobre un vector de aportaciones (default: 0, 500, 1000, 1500, 2000) y retorna un data.frame con impacto de cada una.
- `analyze_retirement_age()` (lineas 363–408): Itera sobre edades 60–70, calculando pension y elegibilidad a cada edad.
- `generate_personalized_message()` (lineas 418–497): Genera mensajes condicionados al resultado (fondo elegible/no, tasa de reemplazo alta/media/baja, aportaciones voluntarias activas/inactivas).

4.5 Tabla de Funciones del Modulo Fondo

Tabla 4.1: Inventario de funciones en `R/fondo_bienestar.R`

Funcion	Linea	Params	Proposito
`check_fondo_eligibility`	25	5	Checklist de 4 condiciones
`calculate_fondo_complement`	107	3	Formula gap-fill
`calculate_pension_with_fondo`	165	9	Orquestador 3 escenarios
`analyze_voluntary_contributions`	315	8	Sensibilidad aportaciones
`analyze_retirement_age`	363	8	Sensibilidad edad
`generate_personalized_message`	418	1	Mensajes personalizados

Capítulo 5

Arquitectura UI: Componentes y Diseno

5.1 Sistema de Diseno CSS

Patron de Software

Patron: Design Tokens via CSS Custom Properties. El archivo `www/styles.css` define un sistema de diseno completo usando CSS custom properties (variables) en las primeras 106 lineas. Todos los colores, sombras, gradientes, espaciados y transiciones se referencian via variables, lo que permite cambiar el tema entero modificando solo el bloque `:root`.

Intuicion de Flujo

El palette "Tropical Vibrant" reemplaza un diseno anterior basado en navy blue (`#1a365d`). La paleta actual usa teal como primario (confianza, estabilidad) y magenta como acento (energia, accion). El fondo dorado-durazno (`#fef7ed`) da calidez sin sacrificar legibilidad.

Referencia de Codigo

`www/styles.css` lineas 9–106 (extracto de tokens):

```
:root {
  --primary: #0f766e;        /* teal */
  --primary-dark: #115e59;
  --secondary: #db2777;      /* magenta */
  --secondary-light: #ec4899;
  --body-bg: #fef7ed;        /* golden-peach */
  --surface: #fffbf0;        /* card background */
  --text: #1e293b;
  --text-muted: #c4a67a;     /* golden muted */
  --border: #f0e6d6;
  --shadow-sm: 0 1px 2px rgba(15,118,110,0.06);
  --shadow-md: 0 4px 12px rgba(15,118,110,0.08);
  --transition: all 0.2s ease;
  --gradient-hero: linear-gradient(135deg, #0f766e 0%, #f97316
     100%);
```

```
}
```

Listing 5.1: Design tokens CSS

5.1.1 Breakpoints Responsivos

Referencia de Codigo

Tres breakpoints en `www/styles.css`:

- `@media (max-width: 992px)`: Grid 2 columnas a 1 columna, wizard header simplificado.
- `@media (max-width: 768px)`: Sliders apilados verticalmente, fuentes reducidas.
- `@media (max-width: 576px)`: Layout full mobile, hero compacto, padding reducido.

5.2 Componentes R de UI

El archivo `R/ui_helpers.R` (1,330 lineas) define todos los componentes de interfaz como funciones R que retornan objetos `tagList` de Shiny.

5.2.1 pension_theme

Patron de Software

Patron: Programmatic Theme. En vez de definir el tema Bootstrap solo en CSS, `pension_theme()` usa `bslib::bs_theme()` para configurar Bootstrap 5 desde R, lo que permite que Shiny genere CSS coherente con el tema.

Referencia de Codigo

`R/ui_helpers.R` lineas 10–45:

```r
pension_theme <- function() {
  bslib::bs_theme(
    version = 5,
    bg = "#fef7ed",
    fg = "#1e293b",
    primary = "#0f766e",
    secondary = "#db2777",
    base_font = bslib::font_google("Inter"),
    heading_font = bslib::font_google("Inter"),
    font_scale = 1.0
  )
}
```

Listing 5.2: Tema Bootstrap 5 programatico

Referencia en `app.R:20`: `theme = pension_theme()`.

5.2.2 Landing Page Components

Referencia de Codigo

Componentes de la pagina de aterrizaje (orden de aparicion):

- `hero_section()` (linea 53): Hero con gradiente, titulo, subtitulo, CTA "Calcular mi pension", link "Quiero entender primero".
- `value_prop_item(icon, title, desc)` (linea 101): Card de propuesta de valor con icono Bootstrap.
- `trust_indicators()` (linea 119): Badges de confianza ("Gratis", "Sin registro", "100% privado").
- `context_section()` (linea 145): Seccion educativa colapsable.
- `pension_timeline()` (linea 197): Timeline visual de la historia del sistema de pensiones mexicano.
- `comparison_table()` (linea 244): Tabla Ley 73 vs Ley 97.
- `control_framework()` (linea 279): Marco verde/amarillo/rojo de que controla el usuario.

5.2.3 Wizard Components

Patron de Software

Patron: Step-Based Wizard. Los componentes del wizard (`wizard_header`, `wizard_panel`) generan la estructura de un wizard de 4 pasos. Los paneles se muestran/ocultan via `shinyjs::show/hide`, y el header se actualiza con clases CSS `.active/.completed/.upcoming`.

Referencia de Codigo

- `wizard_header(step, total)` (linea 344): Genera indicadores de paso con iconos y etiquetas. Clase `.wizard-step` con modificadores.
- `wizard_panel(step, ...)` (linea 384): Wrapper de contenido de cada paso con `id = paste0("step", step, "_panel")`.

Los 4 pasos en `app.R` UI (lineas 180–720):

1. **Datos personales**: fecha nacimiento, genero, edad retiro
2. **Datos laborales**: fecha inicio cotizacion, salario, semanas
3. **AFORE**: seleccion de AFORE, saldo, aportaciones voluntarias
4. **Resultados**: hero, breakdown, sliders de sensibilidad, grafica, reportes

5.2.4 Result Rendering

Patron de Software

Patron: Regime-Dispatched Rendering. El sistema usa dos funciones separadas para renderizar resultados: `render_results_hero()` para Ley 97 y `render_results_hero_ley73()` para Ley 73. Esto refleja que los dos regimenes tienen estructuras de resultado fundamentalmente diferentes.

Intuicion de Flujo

Para Ley 97, el resultado principal es la pension AFORE con complemento del Fondo y el impacto de aportaciones voluntarias. El hero muestra 3 escenarios apilados. Para Ley 73, el resultado principal es la pension definida, con Modalidad 40 como alternativa. No hay Fondo Bienestar — es irrelevante para Ley 73.

Referencia de Codigo

Funciones de renderizado de resultados en `R/ui_helpers.R`:

- `result_card(type, title, value, subtitle)` (linea 453): Card individual de resultado. `type` ∈ {"base", "fondo", "acciones"}.
- `detect_result_scenario(resultado)` (linea 490): Clasifica el resultado para determinar que renderizar.
- `render_results_hero(resultado)` (linea 511): Ley 97 — hero section con pension principal, breakdown (AFORE vs Fondo vs Voluntarias), y tasa de reemplazo. 228 lineas de HTML generation.
- `render_results_hero_ley73(resultado)` (linea 743): Ley 73 — hero con pension definida, desglose de formula (cuantia, incrementos, factor cesantia), y opcion Modalidad 40. 168 lineas.
- `encouragement_message(resultado)` (linea 919): Mensaje motivacional condicionado a la tasa de reemplazo y elegibilidad del Fondo.

5.2.5 Componentes Auxiliares de UI

Referencia de Codigo

Otros componentes en `R/ui_helpers.R`:

- `help_tooltip(text)` (linea 994): Tooltip con icono "?".
- `input_with_help(input, help_text)` (linea 1007): Wrapper input + tooltip.
- `GLOSARIO` (linea 1028): Lista nombrada con 14 terminos definidos.
- `glossary_panel()` (linea 1072): Accordion UI con el glosario.
- `alert_message(type, title, msg)` (linea 1105): Componente de alerta.
- `technical_panel(resultado)` (linea 1148): Accordion con detalles tecnicos del calculo (valores intermedios, parametros usados).
- `replacement_rate_bar(tasa)` (linea 1219): Barra visual de tasa de reemplazo con colores condicionados (verde $> 70\%$, amarillo 40–70%, rojo $< 40\%$).
- `download_section()` (linea 1263): Botones de descarga de reportes.

5.3 Tabla de Componentes UI

Tabla 5.1: Inventario de funciones en `R/ui_helpers.R`

Funcion	Linea	Cat.	Proposito
`pension_theme`	10	Tema	bslib Bootstrap 5 theme
`hero_section`	53	Landing	Hero con gradiente
`value_prop_item`	101	Landing	Card de propuesta de valor

Funcion	**Linea**	**Cat.**	**Proposito**
`trust_indicators`	119	Landing	Badges de confianza
`context_section`	145	Landing	Seccion educativa
`pension_timeline`	197	Landing	Timeline historico
`comparison_table`	244	Landing	Tabla Ley73 vs Ley97
`control_framework`	279	Landing	Marco verde/amarillo/rojo
`wizard_header`	344	Wizard	Indicadores de paso
`wizard_panel`	384	Wizard	Panel de contenido
`result_card`	453	Results	Card de resultado
`detect_result_scenario`	490	Results	Clasificador de resultado
`render_results_hero`	511	Results	Hero Ley 97 (228 lines)
`render_results_hero_ley73`	743	Results	Hero Ley 73 (168 lines)
`encouragement_message`	919	Results	Mensaje motivacional
`help_tooltip`	994	Form	Tooltip "?"
`input_with_help`	1007	Form	Input + tooltip wrapper
`GLOSARIO`	1028	Ref	14 terminos definidos
`glossary_panel`	1072	Ref	Accordion glosario
`alert_message`	1105	Feedback	Componente de alerta
`technical_panel`	1148	Results	Detalles tecnicos accordion
`replacement_rate_bar`	1219	Results	Barra tasa de reemplazo
`download_section`	1263	Results	Botones de descarga

Capítulo 6

Flujo Reactivo del Servidor

Este capitulo mapea el grafo reactivo completo del servidor Shiny en `app.R` (lineas 743–1,951). El servidor gestiona 8 valores reactivos, ∼20 observers, y multiples outputs.

6.1 Estado Reactivo

Patron de Software

Patron: Centralized Reactive State. El servidor define 8 `reactiveVal()`s que actuan como store central. A diferencia de un `reactiveValues()` (que seria un store unico con multiples slots), se usan `reactiveVal()`s individuales para granularidad de invalidacion: modificar `resultados()` no invalida observers que solo dependen de `current_step()`.

Referencia de Codigo

`app.R` lineas 765–784:

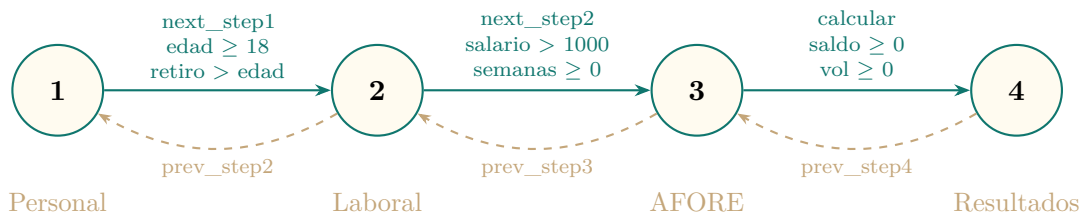
```
current_step <- reactiveVal(1)                    # Paso del wizard (1-4)
resultados <- reactiveVal(NULL)                   # Resultados actuales (con sliders)
resultados_originales <- reactiveVal(NULL)        # Baseline fijo para grafico
regimen_actual <- reactiveVal("ley97")            # Regimen detectado/override
prev_fondo_eligible <- reactiveVal(NULL)          # Tracker para cliff notification
prev_aplico_minimo <- reactiveVal(NULL)           # Tracker para pension minima
fecha_cotizacion_user_edited <- reactiveVal(FALSE)# Flag de edicion manual
regimen_override_active_val <- reactiveVal(FALSE) # Flag de override manual
```

Listing 6.1: Los 8 reactiveVals del servidor

6.2 Transiciones de Estado del Wizard

Patron de Software

Patron: State Machine via Show/Hide. El wizard no usa una libreria de routing — simplemente muestra/oculta paneles con `shinyjs::show/hide`. El estado `current_step()` es la "fuente de verdad" del paso actual. Las transiciones estan protegidas por validacion en cada paso.



Referencia de Codigo

Validaciones por paso:

- **Step 1 → 2** (lineas 831–873): `fecha_nacimiento` no NULL, edad ≥ 18, `edad_retiro` 60–70, retiro > edad actual.
- **Step 2 → 3** (lineas 884–917): `salario_mensual` ≥ 1000, `semanas_cotizadas` ≥ 0, `fecha_inicio_cotizacion` valida y no futura.
- **Step 3 → 4** (lineas 1046–1150): `saldo_afore` ≥ 0, `aportacion_voluntaria` ≥ 0. Dispara calculo completo.

6.3 Auto-fill y Deteccion de Regimen

Patron de Software

Patron: Cascading Defaults with Override. La fecha de nacimiento auto-rellena la fecha de inicio de cotizacion (nacimiento + 18 anos), que a su vez auto-determina el regimen (Ley 73 si < 1997-07-01). Ambos auto-fills tienen flags de override: `fecha_cotizacion_user_edited` y `regimen_override_active_val` previenen que ediciones manuales se sobrescriban.

Referencia de Codigo

Flujo de auto-fill (`app.R`):

```
# 1. Fecha nacimiento -> auto-fill fecha inicio (lineas 931-938)
observeEvent(input$fecha_nacimiento, {
  if (!fecha_cotizacion_user_edited()) {
    default_start <- input$fecha_nacimiento + as.difftime(18*365.25, units="days")
    updateDateInput(session, "fecha_inicio_cotizacion", value = default_start)
  }
```

```r
})

# 2. Track de edicion manual (lineas 941-943)
observeEvent(input$fecha_inicio_cotizacion, {
  if (current_step() >= 2) fecha_cotizacion_user_edited(TRUE)
}, ignoreInit = TRUE)

# 3. Fecha inicio -> regimen automatico (lineas 949-963)
observeEvent(input$fecha_inicio_cotizacion, {
  if (!regimen_override_active_val()) {
    regimen_actual(determinar_regimen(
        input$fecha_inicio_cotizacion))
  }
  # Cross-validacion edad de inicio (lineas 957-962)
})

# 4. Override manual de regimen (lineas 966-977)
observeEvent(input$regimen_manual, {
  if (regimen_override_active_val()) regimen_actual(
      input$regimen_manual)
})
```

Listing 6.2: Cascada fecha nacimiento -> regimen

6.4 Calculo Principal

Patron de Software

Patron: Regime Dispatch. El `observeEvent(input$calcular)` en linea 1046 es el dispatcher central. Segun el regimen detectado, despacha a `calculate_ley73_pension()` (con M40 opcional) o a `calculate_pension_with_fondo()` (que internamente calcula los 3 escenarios). El resultado se almacena en `resultados()` Y en `resultados_originales()`.

Intuicion de Flujo

El patron dual-store es fundamental: `resultados()` se actualiza cada vez que un slider cambia (para actualizar la UI), pero `resultados_originales()` permanece fijo al valor del calculo inicial. Esto permite que la grafica de Plotly muestre una "linea base" inmovil contra la cual comparar los cambios de sensibilidad.

Referencia de Codigo

`app.R` lineas 1046–1150 (estructura):

```r
observeEvent(input$calcular, {
  # Validaciones (lineas 1050-1059)
  edad_actual <- as.numeric(difftime(Sys.Date(),
      input$fecha_nacimiento, ...)) / 365.25

  if (regimen == "ley73") {
```

```
    # Branch Ley 73 (lineas 1068-1108)
    resultado_base <- calculate_ley73_pension(sbc_diario, semanas_al_retiro, edad_retiro)
    resultado_m40 <- calculate_modalidad_40(...)  # si elegible y anios > 0
    res <- list(regimen="ley73", pension_base=resultado_base, pension_m40=resultado_m40)
  } else {
    # Branch Ley 97 (lineas 1110-1125)
    res <- calculate_pension_with_fondo(saldo, salario, edad, ...)
    res$regimen <- "ley97"
  }

  resultados(res)                    # Mutable: actualizado por sliders
  resultados_originales(res)         # Inmutable: baseline para grafico

  # Inicializar sliders con valores actuales (lineas 1140-1143)
  # Transicion visual al paso 4 (lineas 1146-1149)
})
```

Listing 6.3: Dispatch principal de calculo

6.5 Sensibilidad con Debounce

Patron de Software

Patron: Debounced Reactive Chain. Los 5 sliders de sensibilidad pasan por `debounce(300)` antes de alimentar al observer unificado de recalculo. El debounce previene que cada tick del slider dispare un recalculo completo — solo se recalcula 300ms despues del ultimo movimiento.

Intuicion de Flujo

Sin debounce, mover un slider de 1000 a 2000 pesos dispararia ~1000 recalculos (uno por cada peso). Con debounce de 300ms, solo se dispara 1 recalculo al final del movimiento. Esto es critico para performance porque cada recalculo invoca `calculate_pension_with_fondo()` que a su vez llama a `calculate_ley97_pension()` dos veces.

Referencia de Codigo

`app.R` lineas 1174–1179 (debounce) y 1205–1306 (recalculo unificado):

```
# Debounce de 300ms para cada slider (lineas 1174-1179)
vol_debounced <- reactive({ input$slider_voluntaria }) |> debounce(300)
edad_debounced <- reactive({ input$slider_edad }) |> debounce
```

```
    (300)
semanas_debounced <- reactive({ input$slider_semanas }) |>
    debounce(300)
afore_debounced <- reactive({ input$afore_comparar }) |>
    debounce(300)
salario_debounced <- reactive({ input$slider_salario }) |>
    debounce(300)

# Observer unificado (linea 1205)
observe({
  req(resultados_originales())
  res_orig <- resultados_originales()

  # Leer valores debounced con fallback %||% (lineas 1209-1213)
  vol_actual <- vol_debounced() %||%
      res_orig$entrada$aportacion_voluntaria %||% 0
  edad_slider <- edad_debounced() %||%
      res_orig$entrada$edad_retiro
  # ...

  if (res_orig$regimen == "ley97") {
    res <- calculate_pension_with_fondo(...)    # Recalculo
        completo
    # Cliff notifications (lineas 1231-1280)
    resultados(res)                             # Actualizar
        store mutable
  } else {  # ley73
    resultado_base <- calculate_ley73_pension(...)
    resultado_m40 <- calculate_modalidad_40(...)
    resultados(list(...))
  }
})
```

Listing 6.4: Debounce y recalculo unificado

6.6 Impact Labels (Etiquetas de Impacto)

Patron de Software

Patron: Delta Display. Para cada slider, un observer dedicado calcula la diferencia entre el valor original (baseline) y el valor actual (con slider). El resultado se muestra como etiqueta coloreada: verde si positivo, rojo si negativo, gris si neutral.

Referencia de Codigo

`app.R` lineas 1308–1638. Hay 5 observers de impact label:

- `slider_voluntaria` → `vol_impact` (lineas 1309–1355)
- `slider_edad` → `age_impact` (lineas 1358–1420)
- `afore_comparar` → `afore_impact` (lineas 1424–1483)
- `slider_semanas` → `semanas_impact` (lineas 1486–1575)

- `slider_salario` → `salario_impact` (lineas 1579–1638)

Patron comun: obtener pension original de `resultados_originales()`, pension actual de `resultados()`, calcular diferencia y porcentaje, generar HTML con color condicionado.

6.7 Cliff Notifications

Patron de Software

Patron: State Transition Alerts. El sistema detecta cuando un cambio de slider cruza un umbral critico (por ejemplo, perder elegibilidad del Fondo o caer en pension minima) y muestra una notificacion unica. Los `reactiveVals` `prev_fondo_eligible` y `prev_aplico_minimo` rastrean el estado anterior para detectar transiciones.

Referencia de Codigo

`app.R` lineas 1231–1280 (dentro del observer unificado):

```r
# Cliff: perdida de elegibilidad al Fondo (lineas 1232-1248)
prev_eligible <- isolate(prev_fondo_eligible())
new_eligible <- res$con_fondo$elegible
if (!is.null(prev_eligible) && prev_eligible != new_eligible) {
  if (!new_eligible) {
    showNotification("Con estos cambios pierdes elegibilidad...
        ", type="warning")
  } else {
    showNotification("Con estos cambios ganas elegibilidad!",
        type="message")
  }
}
prev_fondo_eligible(new_eligible)

# Cliff: caida/salida de pension minima (lineas 1254-1278)
# Similar: compara prev_aplico_minimo con nuevo estado
```

Listing 6.5: Deteccion de cliff notifications

6.8 Grafica Plotly

Patron de Software

Patron: Dual-Dispatch Chart. La grafica usa dos modos completamente diferentes segun el regimen: barras horizontales para Ley 73, y 3 trazos de linea superpuestos para Ley 97 (trayectoria de saldo AFORE a lo largo del tiempo).

Referencia de Codigo

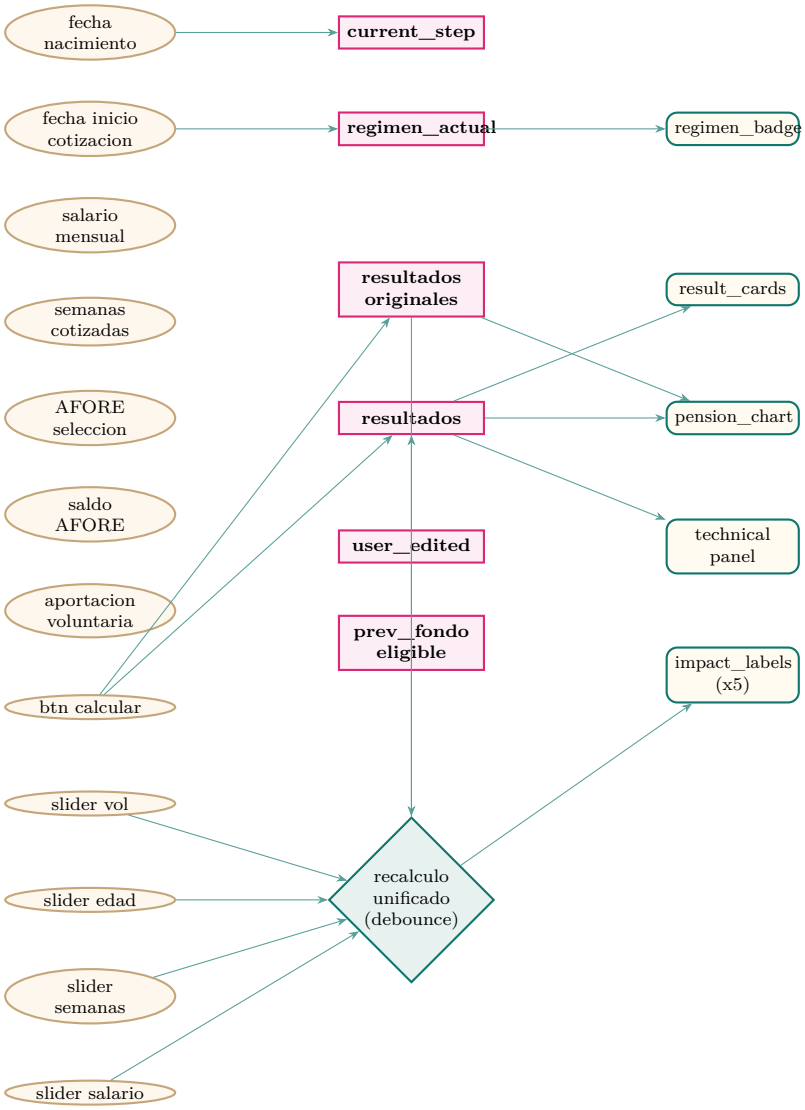
`app.R` lineas 1658–1777. Estructura:

```r
output$pension_chart <- plotly::renderPlotly({
  req(resultados())
  if (resultados()$regimen == "ley73") {
    # Barras horizontales: pension base vs M40 (lineas 1667-1720)
    plot_ly(type="bar", orientation="h", ...)
  } else {
    # Ley 97: 3 trazos superpuestos (lineas 1725-1770)
    # Trazo 1: baseline (dorado punteado) - de resultados_originales()
    # Trazo 2: con cambios (teal discontinuo) - de resultados()
    # Trazo 3: con aportaciones (teal solido) - de resultados() con vol > 0
    plot_ly() |>
      add_trace(trayectoria_original, line=list(dash="dot", color="#c4a67a")) |>
      add_trace(trayectoria_actual, line=list(dash="dash", color="#0f766e")) |>
      add_trace(trayectoria_acciones, line=list(color="#0f766e"))
  }
})
```

Listing 6.6: Grafica Plotly dual-mode

Los 3 trazos usan trayectorias de `resultados_originales()` (baseline) y `resultados()` (con sliders), lo que permite ver el impacto visual de cada cambio.

6.9 Diagrama del Grafo Reactivo Completo



Capítulo 7

Pipeline de Generacion de Documentos

El archivo `R/document_generators.R` (1,667 lineas) implementa la generacion de reportes HTML y PDF/RMarkdown.

7.1 Arquitectura de Reportes

Patron de Software

Patron: Dual Format Generation. Cada tipo de reporte tiene dos implementaciones: una directa en HTML (para visualizacion rapida en navegador) y una en RMarkdown (para compilacion a PDF via pandoc/LaTeX). La version HTML es la preferida porque no requiere dependencias externas.

Intuicion de Flujo

El pipeline de reportes es: usuario clickea boton → se genera HTML en un archivo temporal en `www/` → JavaScript abre una nueva pestana del navegador → cuando la sesion termina, se borran los archivos temporales. La carpeta `www/` se usa porque Shiny sirve su contenido como archivos estaticos accesibles via URL.

Referencia de Codigo

Funciones de generacion en `R/document_generators.R`:

- `get_document_css()` (linea 10): CSS compartido para todos los reportes. Usa la misma paleta Tropical Vibrant.
- `generate_basic_report(resultado, output_file)` (linea 1047): Reporte basico HTML — resumen de pension, datos de entrada, resultado principal.
- `generate_technical_report(resultado, output_file)` (linea 756): Reporte tecnico HTML — incluye todos los valores intermedios del calculo.
- `generate_methodology_html(output_file)` (linea 1200): Documento de metodologia estatico (no depende de resultado).
- `generate_summary_report(resultado, output_file)` (linea 1545): Resumen ejecutivo HTML.
- `generate_basic_rmd(resultado, output_file)` (linea 410): RMarkdown basico.

- `generate_technical_rmd(resultado, output_file)` (linea 523): RMarkdown tecnico.
- `generate_summary_rmd(resultado, output_file)` (linea 1444): RMarkdown resumen.
- `generate_methodology_pdf(output_file)` (linea 341): Metodologia RMarkdown/PDF.

7.2 Ciclo de Vida de Archivos Temporales

Patron de Software

Patron: Session-Scoped Temp Files. Los reportes se generan como archivos temporales en el directorio `www/` del servidor. Cada archivo recibe un nombre unico via `tempfile()`. Cuando la sesion de Shiny termina, un callback registrado con `session$onSessionEnded` limpia los archivos.

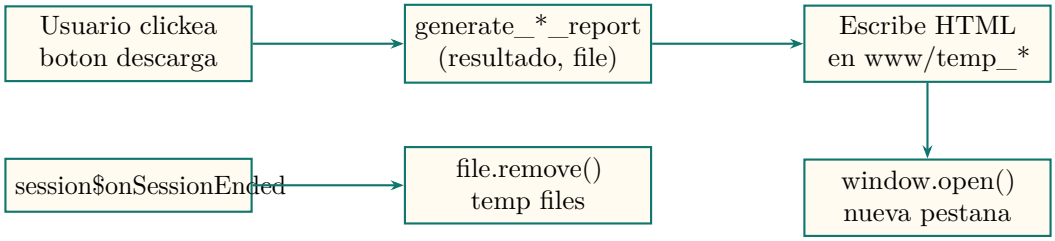
Referencia de Codigo

`app.R` lineas 1868–1932 (patron de generacion y limpieza):

```r
# Generacion (dentro de observeEvent del boton de descarga)
temp_file <- tempfile(tmpdir = "www", fileext = ".html")
generate_basic_report(resultados(), temp_file)

# Abrir en nueva pestana
shinyjs::runjs(paste0('window.open("', basename(temp_file), '",
    "_blank")'))

# Limpieza al cerrar sesion
session$onSessionEnded(function() {
  temp_files <- list.files("www", pattern = "^file.*\\.html$",
      full.names = TRUE)
  file.remove(temp_files)
})
```

Listing 7.1: Ciclo de vida de archivos temporales

7.2.1 Diagrama del Pipeline de Reportes



7.3 Tabla de Funciones de Documentos

Tabla 7.1: Inventario de funciones en `R/document_generators.R`

Funcion	**Linea**	**Formato**	**Estatico?**	**Proposito**
`get_document_css`	10	CSS	–	CSS compartido
`generate_methodology_pdf`	341	RMD	Si	Metodologia PDF
`generate_basic_rmd`	410	RMD	No	Reporte basico PDF
`generate_technical_rmd`	523	RMD	No	Reporte tecnico PDF
`generate_technical_report`	756	HTML	No	Reporte tecnico rapido
`generate_basic_report`	1047	HTML	No	Reporte basico rapido
`generate_methodology_html`	1200	HTML	Si	Metodologia HTML
`generate_summary_rmd`	1444	RMD	No	Resumen PDF
`generate_summary_report`	1545	HTML	No	Resumen rapido

Capítulo 8

Arquitectura de Pruebas

8.1 Estructura del Test Suite

Patron de Software

Patron: Isolated Unit Tests without Shiny. Las pruebas unitarias cargan los datos y funciones directamente (sin arrancar un server Shiny) replicando la carga de `global.R` manualmente. Esto permite ejecutar pruebas rapidas sin overhead de la UI.

Intuicion de Flujo

El archivo principal de tests tiene 95 pruebas agrupadas en 14 secciones (A–N). Cada seccion prueba una funcion o un grupo de funciones relacionadas. El orden sigue la dependencia del codigo: primero se prueban las funciones de datos, luego las de calculo, luego las de integracion.

Referencia de Codigo

`tests/testthat/test_calculations.R` lineas 14–64 (preambulo):

```r
library(testthat)
library(dplyr)

# Cargar datos directamente (lineas 22-25)
articulo_167_tabla <- read.csv("../../data/articulo_167_tabla.
    csv")
afore_data <- read.csv("../../data/afore_comisiones.csv")
uma_data <- read.csv("../../data/uma_historico.csv")
salario_minimo_data <- read.csv("../../data/salario_minimo.csv"
    )

# Constantes (replicando global.R, lineas 30-45)
ANIO_ACTUAL <- 2025
UMA_DIARIA_2025 <- 113.14
SM_DIARIO_2025 <- 278.80
# ...
```

```r

# Source funciones (lineas 50-55)
source("../../R/data_tables.R")
source("../../R/calculations.R")
source("../../R/fondo_bienestar.R")

# Helper: ignore name attributes (lineas 69-71)
expect_num <- function(actual, expected, tolerance = 0.01) {
  expect_equal(as.numeric(actual), expected, tolerance =
      tolerance)
}
```

Listing 8.1: Preambulo de tests: carga sin Shiny

8.2 Secciones de Prueba

Tabla 8.1: 95 pruebas en 14 secciones

Seccion	Funcion Objetivo	Tests	Alcance
A	`lookup_articulo_167`	6	Lookup exacto, limites, tope
B	`calculate_ley73_pension`	10	8 pasos, minima, cesantia
C	`project_afore_balance`	7	FV, trayectoria, r=0
D	`calculate_aportacion_obligatoria`	6	Tasas por ano, tope SBC
E	`calculate_retiro_programado`	6	Pension, minima, genero
F	`calculate_ley97_pension`	6	Pipeline completo
G	`calculate_modalidad_40`	7	Costo, beneficio, ROI
H	`check_fondo_eligibility`	8	4 condiciones, combinaciones
I	`calculate_fondo_complement`	4	Complemento, tope, cero
J	`calculate_pension_with_fondo`	4	Integracion 3 escenarios
K	Helpers	12	`format_currency`, `%||%`, `get_uma`, etc.
L	Sensibilidad	8	Impacto de vol, edad, semanas
M	Pipeline completo	16	22+ perfiles QA simulados
N	`validar_consistencia_fechas`	3	Cross-date validation

8.3 Patron expect_num

Patron de Software

Patron: Named Vector Unwrapping. Varias funciones R retornan vectores nombrados (por ejemplo, `FACTORES_CESANTIA["65"]` retorna `c("65" = 1.0)`). El helper `expect_num()` llama a `as.numeric()` para eliminar el atributo de nombre antes de comparar, evitando falsos negativos por metadata de nombres.

Referencia de Codigo

`tests/testthat/test_calculations.R` lineas 69–71:

```
expect_num <- function(actual, expected, tolerance = 0.01) {
  expect_equal(as.numeric(actual), expected, tolerance =
     tolerance)
}
```

Listing 8.2: Helper expect_num

Uso tipico: `expect_num(resultado$pension_mensual, 12345.67)`

8.4 Otros Archivos de Prueba

Referencia de Codigo

- `tests/integration_test.R`: Prueba el pipeline de sensibilidad frontend-backend simulando cambios de sliders.
- `tests/qa_test_profiles.R`: 22+ perfiles de usuario simulados con datos realistas (nombres, edades, salarios tipicos de Mexico).
- `tests/edge_case_tests.R`: Casos limite: salario = 0, semanas = 0, edad = 18, saldo negativo, etc.
- `tests/validate_results_responsiveness.R`: Verifica que la UI responde correctamente a cambios de regimen y sliders.
- `tests/testthat.R`: Runner que ejecuta el suite via `testthat::test_dir()`.

Ejecutar: `Rscript tests/testthat.R`

Appendix A

Indice Completo de Funciones

Este apendice lista todas las funciones publicas del sistema ordenadas por archivo fuente, con firma completa y linea de inicio.

A.1 global.R (153 lineas)

Funcion	Linea	Firma
`%||%`	85	`function(x, y)`
`format_currency`	90	`function(x)`
`format_currency_latex`	95	`function(x)`
`format_percent`	100	`function(x)`
`format_percent_latex`	105	`function(x)`
`get_uma`	110	`function(anio)`
`get_salario_minimo`	119	`function(anio)`
`get_umbral_fondo`	128	`function(anio)`

A.2 R/data_tables.R (244 lineas)

Funcion	Linea	Firma
`lookup_articulo_167`	12	`function(grupo_salarial, tabla)`
`get_afore_names`	38	`function()`
`get_afore_comision`	46	`function(afore_nombre, anio)`
`get_afore_irn`	63	`function(afore_nombre)`
`get_all_afores`	73	`function()`
`get_esperanza_vida`	89	`function(edad, genero)`
`validar_entrada`	155	`function(edad, semanas, sbc, fecha)`
`determinar_regimen`	208	`function(fecha_primera_cotizacion)`
`validar_consist...fechas`	228	`function(fecha_nac, fecha_inicio)`

A.3 R/calculations.R (671 lineas)

Funcion	Linea	Firma
calculate_ley73_pension	21	function(sbc, semanas, edad, sm, tipo)
project_afore_balance	118	function(saldo, aport, anios, rend, com, tray)
calculate_aport_oblig	188	function(salario, anio)
calculate_retiro_prog	259	function(saldo, edad, genero)
calculate_ley97_pension	313	function(saldo, sal, edad, ret, sem, gen, vol, afore, esc)
calculate_modalidad_40	412	function(pension, sbc, sbc_m40, sem, sem_m40, edad, ret)
calculate_all_scenarios	498	function(regimen, saldo, sal, sbc, edad, ret, sem, gen, vol
compare_afores	624	function(saldo, sal, anios, afore)

A.4 R/fondo_bienestar.R (497 lineas)

Funcion	Linea	Firma
check_fondo_eligibility	25	function(regimen, edad, semanas, sbc, anio)
calculate_fondo_complement	107	function(pension, sbc, elegibilidad)
calculate_pension_with_fondo	165	function(saldo, sal, edad, ret, sem, gen, vol, afore,
analyze_vol_contributions	315	function(saldo, sal, edad, ret, sem, gen, afore, aport
analyze_retirement_age	363	function(saldo, sal, edad, sem, gen, vol, afore, edade
generate_personalized_msg	418	function(resultado)